Problem_3

a)

Assuming there is no bias:

$$h_\theta(\boldsymbol{x}) = \boldsymbol{\theta^T x} = \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n \quad (1)$$

From equation (1), it can be seen that no matter the values of the theta if the vector $\boldsymbol{x} = \mathbf{0}$, the value of $h_\theta(\boldsymbol{x})$ will always be zero and therefore it will pass through the origin. If a bias is added to $h_\theta(\boldsymbol{x})$ and the value of the bias is not zero, then at $\boldsymbol{x} = \mathbf{0}$, $h_\theta(\boldsymbol{x})$ doesn't equal to zero and therefore $h_\theta(\boldsymbol{x})$ doesn't pass through the origin.

To describe this geometrically in $\mathbb{R}^2$, assume $(h_\theta(\boldsymbol{x}) = \theta_0 + \theta_1 x_1)$, it can be clearly seen that if θ_0 doesn't exist then we get the equation of a line in 2D space that passes through the origin with slope equals to θ_1. If θ_0 exists however, we get the same line but shifted θ_0 units in the y-axis.

b)

Minimizing the residuals without absolute or square functions is simply a bad idea because residuals with positive values will cancel residuals with negative value, and because of this the value of the loss can be very small and still the model doesn't fit the data.

For gradient-based optimization algorithms, we need to find the gradient of the loss with respect to the model's parameters, finding the gradients requires that the loss function to be differentiable at every point in the parameters space. For the squared loss this is true, however for absolute loss, the loss function is not differentiable at each point because there are sharp edges in the function.

c)

$$J(\boldsymbol{\theta}) = \frac{1}{2}\sum_{i}^{n}(\boldsymbol{\theta^T x_i} - y_i)^2 = \frac{1}{2}\|\boldsymbol{X\theta} - \boldsymbol{y}\|^2 \quad (2)$$

Where $\boldsymbol{X} \in \mathbb{R}^{n\times d}$ has rows $\boldsymbol{x_i^T}$ and $\boldsymbol{y} \in \mathbb{R}^n$

Finding the gradient of $J(\boldsymbol{\theta})$:

$$\nabla_\theta J(\boldsymbol{\theta}) = \boldsymbol{X^T}(\boldsymbol{X\,\theta} - \boldsymbol{y}) \quad (3)$$

Finding the Hessian of $J(\boldsymbol{\theta})$:

$$\nabla_\theta^2 J(\boldsymbol{\theta}) = \boldsymbol{X^T X} \quad (4)$$

For any vector $\boldsymbol{v} \in \mathbb{R}^d$:

$$\boldsymbol{v^T}(\boldsymbol{X^T X})\boldsymbol{v} = \|\boldsymbol{Xv}\|^2 \geq \boldsymbol{0} \quad (5)$$

It can be seen from equation (5) that the Hessian matrix is positive semidefinite, hence $J(\boldsymbol{\theta})$ is convex.

$J(\theta)$ Is strictly convex if the Hessian is positive definite:

$$\boldsymbol{v^T}(\boldsymbol{X^T X})\boldsymbol{v} = \|\boldsymbol{Xv}\|^2 > \boldsymbol{0} \quad (6)$$

This occurs if the columns of X are linearly independent or in other words $rank(X) = d$

d)

$$\nabla_\theta J(\boldsymbol{\theta}) = \begin{bmatrix} \frac{\partial}{\partial \theta_1} \frac{1}{2} \sum_i^n \left(\boldsymbol{\theta^T x^{(i)}} - y_i\right)^2 \\ \vdots \\ \frac{\partial}{\partial \theta_n} \frac{1}{2} \sum_i^n \left(\boldsymbol{\theta^T x^{(i)}} - y_i\right)^2 \end{bmatrix} = \begin{bmatrix} \sum_i^n \left(\boldsymbol{\theta^T x^{(i)}} - y_i\right) x_1^{(i)} \\ \vdots \\ \sum_i^n \left(\boldsymbol{\theta^T x^{(i)}} - y_i\right) x_n^{(i)} \end{bmatrix} \quad (7)$$

Where:

$\boldsymbol{x^{(i)}}$: is a vector representing the i_{th} example of dataset.

$x_n^{(i)}$: is the n_{th} element of the i_{th} example.

Writing the batch gradient descent formula using the gradient in equation (3):

For epoch $1 \cdots T$:

$$\begin{bmatrix} \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} \leftarrow \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} - \alpha \begin{bmatrix} \sum_i^n \left(\boldsymbol{\theta^T x^{(i)}} - y_i\right) x_1^{(i)} \\ \vdots \\ \sum_i^n \left(\boldsymbol{\theta^T x^{(i)}} - y_i\right) x_n^{(i)} \end{bmatrix} \quad (8)$$

e)

Applying batch gradient descent algorithm with the following dataset examples:

$\left(\boldsymbol{x^{(1)}}, y^{(1)}\right) = ([1,2], 5), \quad \left(\boldsymbol{x^{(2)}}, y^{(2)}\right) = ([1, -1], 0)$

Assuming initial value for $\boldsymbol{\theta}^{(0)} = [0,0]$, equation (9) and equation (10) represent the values of the gradient in each row respectively.

$$\sum_i^n \left(\boldsymbol{\theta^T x^{(i)}} - y_i\right) x_1^{(i)} = \left([0 \quad 0] \begin{bmatrix} 1 \\ 2 \end{bmatrix} - 5\right) * 1 + \left([0 \quad 0] \begin{bmatrix} 1 \\ -1 \end{bmatrix} - 0\right) * 1 = -5 \quad (9)$$

$$\sum_i^n \left(\boldsymbol{\theta^T x^{(i)}} - y_i\right) x_2^{(i)} = \left([0 \quad 0] \begin{bmatrix} 1 \\ 2 \end{bmatrix} - 5\right) * 2 + \left([0 \quad 0] \begin{bmatrix} 1 \\ -1 \end{bmatrix} - 0\right) * -1 = -10 \quad (10)$$

Using the gradient from the two equations above to solve for $\boldsymbol{\theta}^{(1)}$:

$$\boldsymbol{\theta}^{(1)} = \boldsymbol{\theta}^{(0)} - \alpha \nabla_{\theta} J(\boldsymbol{\theta}) \tag{11}$$

$$\begin{bmatrix} \theta_0^{(1)} \\ \theta_1^{(1)} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 * \begin{bmatrix} -5 \\ -10 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix} \tag{12}$$

f)

Applying stochastic gradient descent algorithm with the following dataset examples:

$\left(\boldsymbol{x}^{(1)}, y^{(1)}\right) = ([1,2], 5), \quad \left(\boldsymbol{x}^{(2)}, y^{(2)}\right) = ([1, -1], 0)$

Assuming initial value for $\boldsymbol{\theta}^{(0)} = [0,0]$, equation (13) and equation (14) represent the values of the gradient in each row respectively.

When $i = 1$

$$\left(\boldsymbol{\theta}^{\boldsymbol{T}} \boldsymbol{x}^{(i)} - y_i\right) x_1^{(i)} = \left([0 \quad 0] \begin{bmatrix} 1 \\ 2 \end{bmatrix} - 5\right) * 1 = -5 \tag{13}$$

$$\left(\boldsymbol{\theta}^{\boldsymbol{T}} \boldsymbol{x}^{(i)} - y_i\right) x_2^{(i)} = \left([0 \quad 0] \begin{bmatrix} 1 \\ 2 \end{bmatrix} - 5\right) * 2 = -10 \tag{14}$$

Using the gradient from the two equations above to solve for $\boldsymbol{\theta}^{(1)}$:

$$\boldsymbol{\theta}^{(1)} = \boldsymbol{\theta}^{(0)} - \alpha \nabla_{\theta} J(\boldsymbol{\theta}) \tag{15}$$

$$\begin{bmatrix} \theta_0^{(1)} \\ \theta_1^{(1)} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 * \begin{bmatrix} -5 \\ -10 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix} \tag{16}$$

When $i = 2$

$$\left(\boldsymbol{\theta^T x^{(i)}} - y_i\right)x_1^{(i)} = \left([0.5 \quad 1]\begin{bmatrix}1\\-1\end{bmatrix} - 0\right) * 1 = -0.5 \tag{17}$$

$$\left(\boldsymbol{\theta^T x^{(i)}} - y_i\right)x_2^{(i)} = \left([0.5 \quad 1]\begin{bmatrix}1\\-1\end{bmatrix} - 0\right) * -1 = 0.5 \tag{18}$$

Using the gradient from the two equations above to solve for $\boldsymbol{\theta}^{(1)}$:

$$\boldsymbol{\theta}^{(2)} = \boldsymbol{\theta}^{(1)} - \alpha\nabla_\theta J(\boldsymbol{\theta}) \tag{19}$$

$$\begin{bmatrix}\theta_0^{(2)}\\\theta_1^{(2)}\end{bmatrix} = \begin{bmatrix}0.5\\1\end{bmatrix} - 0.1 * \begin{bmatrix}-0.5\\0.5\end{bmatrix} = \begin{bmatrix}0.55\\0.95\end{bmatrix} \tag{20}$$

g)

Plotting $J(\boldsymbol{\theta})$ against θ_0 and θ_1 in the 3D space results in paraboloid shape, taking a cross-section at a given height where $J(\boldsymbol{\theta})$ is constant, we get a contour in the shape of an ellipse.

Gradient direction is perpendicular to these contours.

Choosing α too large may result in overshooting from the loss minimum and the model will become oscillating about it. If choosing α reasonably small, the training will be more stable and chances of overshooting will decrease significantly.